

Les Méthodes d'Ensemble

Combiner les Modèles pour de Meilleures Prédictions

Plan du Cours

- 1 Voting Classifier
- 2 Bagging et Pasting
- 3 Random Forest
- 4 Boosting
- 5 AdaBoost
- 6 Stacking

Le Principe Fondamental

Les **méthodes d'ensemble** combinent plusieurs modèles (appelés *apprenants faibles*) pour créer un modèle plus performant.

La Sagesse des Foules

Un groupe de modèles "moyens" peut souvent surpasser un seul modèle "expert".

Demander l'avis de 1000 personnes sur le poids d'un bœuf donne une meilleure estimation que l'avis d'un seul expert.

(à condition que : les modèles doivent être **diversifiés** et faire des erreurs **indépendantes**.)

Pourquoi les Ensembles Fonctionnent ?

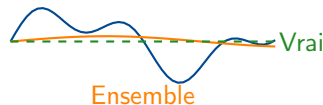
Réduction de la Variance

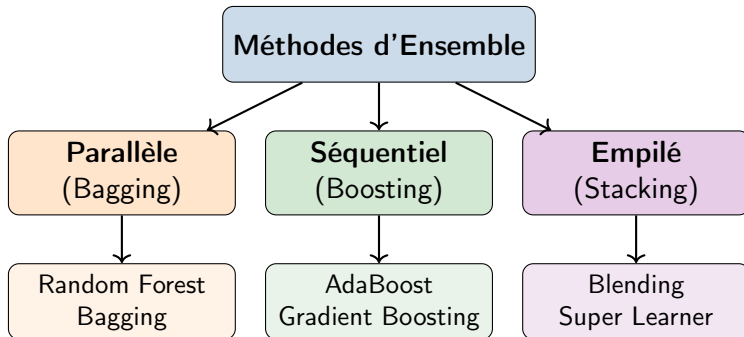
En moyennant plusieurs modèles, on **lisse** les fluctuations individuelles.

Si chaque modèle a une variance σ^2 et qu'ils sont indépendants :

$$\text{Var} \left(\frac{1}{n} \sum_{i=1}^n h_i \right) = \frac{\sigma^2}{n}$$

Un seul modèle





Partie 1

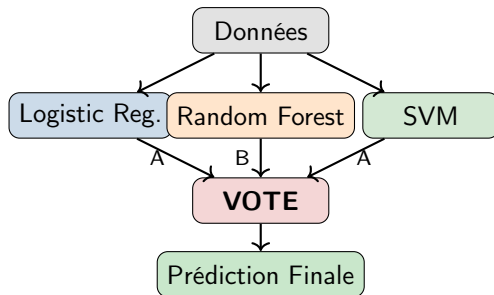
Voting Classifier

Combiner différents modèles par vote

Voting Classifier : Le Principe

Définition

Le **Voting Classifier** agrège les prédictions de plusieurs classifieurs différents et prédit la classe la plus votée.



Hard Voting vs Soft Voting

Hard Voting (Vote Majoritaire)

Chaque classifieur vote pour une classe.

Prédiction : Classe avec le plus de votes.

$$\hat{y} = \text{mode}\{h_1(x), h_2(x), \dots, h_n(x)\}$$

Exemple :

- LR : Classe A
- RF : Classe B
- SVM : Classe A

⇒ **Prédiction : Classe A**

Soft Voting (Vote Pondéré)

Moyenne les **probabilités** de chaque classifieur.

$$\hat{y} = \arg \max_k \frac{1}{n} \sum_{i=1}^n P_i(y = k|x)$$

Exemple :

	A	B
LR	0.6	0.4
RF	0.3	0.7
SVM	0.8	0.2
Moy.	0.57	0.43

Pourquoi Soft Voting est Généralement Meilleur ?

- Prend en compte la **confiance** de chaque classifieur
- Un classifieur très confiant (0.99) pèse plus qu'un incertain (0.51)
- Exploite plus d'information que le simple label prédit

Partie 2

Bagging et Pasting

Diversifier par échantillonnage des données

Observation

Le Voting Classifier utilise des modèles **différents** sur les **mêmes données**.

Alternative : Utiliser le **même modèle** sur des **données différentes** !

L'idée

Créer de la diversité en entraînant chaque modèle sur un **sous-ensemble différent** des données d'entraînement.

Question : Comment créer ces sous-ensembles ?

Bagging : Bootstrap Aggregating

Définition (Leo Breiman, 1996)

Le **Bagging** entraîne chaque modèle sur un échantillon **bootstrap** des données.

Échantillonnage Bootstrap

Tirage aléatoire **avec remise** de n exemples parmi n .

- Certains exemples apparaissent **plusieurs fois**
- D'autres n'apparaissent **jamais** (environ 37%)

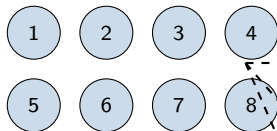
Probabilité d'Exclusion

La probabilité qu'un exemple soit absent d'un échantillon bootstrap :

$$P(\text{absent}) = \left(1 - \frac{1}{n}\right)^n \xrightarrow{n \rightarrow \infty} e^{-1} \approx 0.368$$

Visualisation du Bootstrap

Données originales



Échantillons Bootstrap

Bootstrap 1 :



Bootstrap 2 :



Bootstrap 3 :



Chaque échantillon a la même taille que l'original, mais avec des répétitions.

Bagging vs Pasting

Bagging

Avec remise (bootstrap)

- Plus de diversité
- Chaque modèle voit 63% des données
- Permet l'estimation OOB
- **Plus utilisé en pratique**

Pasting

Sans remise

- Moins de diversité
- Chaque exemple utilisé une fois par modèle
- Échantillons plus petits
- Moins courant

En Pratique

Le Bagging introduit plus de diversité $\Rightarrow$ meilleure réduction de variance.

Le biais peut augmenter légèrement, mais c'est généralement un bon compromis.

L'Algorithme de Bagging

Entraînement

Pour $b = 1, 2, \dots, B$:

- 1 Tirer un échantillon bootstrap S_b de taille n
- 2 Entraîner un modèle h_b sur S_b

Prédiction

Classification : Vote majoritaire

$$\hat{y} = \text{mode}\{h_1(x), h_2(x), \dots, h_B(x)\}$$

Régression : Moyenne

$$\hat{y} = \frac{1}{B} \sum_{b=1}^B h_b(x)$$

Out-of-Bag (OOB) Evaluation

L'Idee (Géniale)

37% des données ne sont pas utilisées pour chaque modèle. On peut les utiliser pour **valider** ce modèle !

OOB Score

Pour chaque exemple x_i :

- 1 Identifier les modèles où x_i était **out-of-bag**
- 2 Faire prédire ces modèles sur x_i
- 3 Agréger leurs prédictions

L'OOB score est la performance moyenne sur ces prédictions.

Partie 3

Random Forest

La forêt d'arbres aléatoires

Random Forest : Bagging Amélioré

Définition (Leo Breiman, 2001)

Le **Random Forest** est un ensemble d'arbres de décision entraînés par Bagging, avec une source **supplémentaire** de randomisation.

La Double Randomisation

- 1 **Bootstrap** : Chaque arbre est entraîné sur un échantillon bootstrap
- 2 **Feature Sampling** : À chaque nœud, seul un **sous-ensemble aléatoire** de caractéristiques est considéré

Cette double randomisation crée des arbres **encore plus diversifiés**.

Le Problème du Bagging Standard

Avec des arbres entraînés sur bootstrap, si une feature domine, **tous les arbres l'utiliseront** à la racine.

⇒ Arbres trop similaires, diversité limitée.

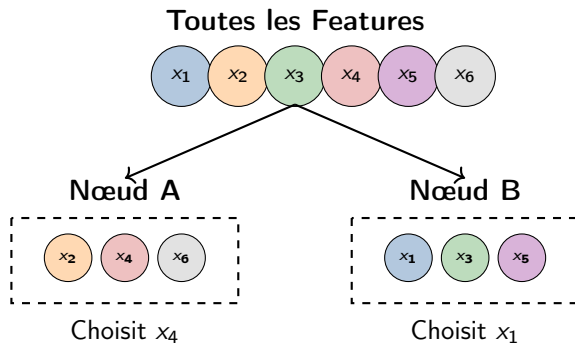
Solution : Random Subspace

À chaque division, on ne considère que m features choisies aléatoirement parmi les p disponibles.

Valeurs typiques de m :

- Classification : $m = \sqrt{p}$
- Régression : $m = p/3$

Visualisation du Feature Sampling



Chaque nœud ne voit qu'un sous-ensemble aléatoire de features.

L'Algorithme Random Forest

Entraînement

Pour $b = 1, 2, \dots, B$ arbres :

- ❶ Tirer un échantillon bootstrap S_b
- ❷ Entraîner un arbre h_b sur S_b avec :
 - À chaque nœud : sélectionner m features aléatoirement
 - Trouver la meilleure division parmi ces m features
 - Faire grandir l'arbre **jusqu'au bout** (pas de pruning)

Prédiction

Classification : Vote majoritaire des B arbres

Régression : Moyenne des B prédictions

Pourquoi Random Forest Fonctionne ?

Arbres Individuels

- Profonds $\Rightarrow$ **Faible biais**
- Non élagués $\Rightarrow$ **Haute variance**
- Légèrement différents

Ensemble

- Conserve le faible biais
- **Réduit la variance** par moyennage
- Plus robuste aux outliers

Random Forest est souvent cité comme le modèle de référence pour commencer un problème de ML

Partie 4

Boosting

Apprendre de ses erreurs

Boosting : L'Idée

Philosophie

Contrairement au Bagging (parallèle), le **Boosting** entraîne les modèles **séquentiellement**. Chaque nouveau modèle tente de **corriger les erreurs** des précédents.

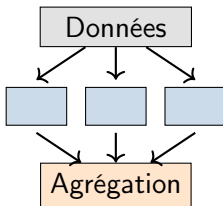


Conséquence

Les modèles **ne peuvent pas** être entraînés en parallèle.

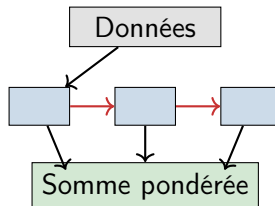
Bagging vs Boosting

BAGGING



Modèles **indépendants**
Réduit la **variance**

BOOSTING



Modèles **séquentiels**
Réduit le **bias**

Weak Learners : La Force des Faibles

Définition : Apprenant Faible

Un modèle légèrement meilleur que le hasard.

Le Miracle du Boosting

En combinant suffisamment d'apprenants faibles de manière intelligente, on peut créer un **apprenant fort** arbitrairement précis !

Pourquoi des Apprenants Faibles ?

- **Haut biais** : Ne capturent pas toute la complexité individuellement
- **Faible variance** : Stables et prévisibles
- En les combinant, on réduit progressivement le biais

Formule Générale

La prédiction finale est une **somme pondérée** des apprenants faibles :

$$H(x) = \sum_{t=1}^T \alpha_t \cdot h_t(x)$$

où α_t est le poids du modèle h_t .

$$H(x) = \underbrace{h_1}_{\alpha_1} + \underbrace{h_2}_{\alpha_2} + \dots + \underbrace{h_T}_{\alpha_T}$$

Correction d'Erreurs : Deux Approches

1. Pondération (AdaBoost)

- On garde les mêmes cibles y
- On donne plus de **poids** aux exemples mal classés
- Le modèle se concentre sur les erreurs

2. Résidus (Gradient Boosting)

- On garde les mêmes poids
- On change la cible : le nouveau modèle apprend les **erreurs** (résidus)
- $h_{t+1}(x) \approx y - H_t(x)$

L'Écosystème Moderne

XGBoost, LightGBM et CatBoost sont des implémentations optimisées du **Gradient Boosting**.

Ils dominent souvent les compétitions sur données tabulaires.

Partie 5

AdaBoost

Adaptive Boosting

Historique

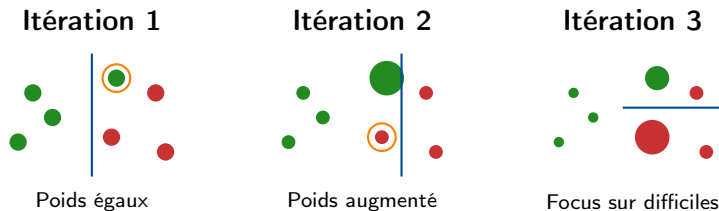
Introduit par **Freund & Schapire (1995)**.

Premier algorithme de Boosting pratique et théoriquement fondé.

L'idée

- 1 Chaque exemple a un **poids** w_i
- 2 Après chaque itération, **augmenter** les poids des exemples mal classés
- 3 Le modèle suivant se **concentre** sur les cas difficiles

Visualisation : Évolution des Poids



La taille des points représente leur poids. Chaque classifieur se concentre sur les erreurs précédentes.

L'Algorithme AdaBoost : Cas Classification Binaire

Initialisation ($y \in \{-1, +1\}$)

$w_i^{(1)} = \frac{1}{n}$ pour tous les exemples (poids uniformes)

Pour $t = 1, 2, \dots, T$:

- 1 Entraîner h_t sur les données pondérées par $w^{(t)}$
- 2 Calculer l'erreur pondérée : $\epsilon_t = \sum_{i: h_t(x_i) \neq y_i} w_i^{(t)}$
- 3 Calculer le poids du modèle : $\alpha_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$
- 4 Mettre à jour les poids :

$$w_i^{(t+1)} = w_i^{(t)} \cdot \exp(-\alpha_t \cdot y_i \cdot h_t(x_i))$$

- 5 Normaliser : $w_i^{(t+1)} \leftarrow \frac{w_i^{(t+1)}}{\sum_j w_j^{(t+1)}}$

Formule

$$w_i^{(t+1)} \propto w_i^{(t)} \cdot \exp(-\alpha_t \cdot y_i \cdot h_t(x_i))$$

Cas 1 : Classification correcte ($y_i \cdot h_t(x_i) = +1$)

$$w_i^{(t+1)} \propto w_i^{(t)} \cdot e^{-\alpha_t}$$

Le poids **diminue** ($e^{-\alpha_t} < 1$ si $\alpha_t > 0$)

Cas 2 : Classification incorrecte ($y_i \cdot h_t(x_i) = -1$)

$$w_i^{(t+1)} \propto w_i^{(t)} \cdot e^{+\alpha_t}$$

Le poids **augmente** ($e^{+\alpha_t} > 1$)

⇒ Les exemples difficiles deviennent **plus importants** !

Classification Binaire

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t \cdot h_t(x) \right)$$

Interprétation

C'est un vote pondéré où :

- Chaque modèle h_t vote $+1$ ou -1
- Son vote est pondéré par α_t (sa "fiabilité")
- On prend le signe de la somme

Partie 6

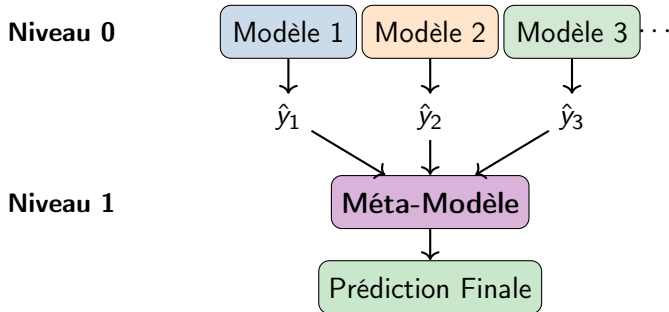
Stacking

Empiler les modèles

Stacking : L'Empilement de Modèles

Définition

Le **Stacking** (ou Stacked Generalization) entraîne un **méta-modèle** à combiner les prédictions de plusieurs modèles de base.



Stacking vs Voting

Voting

- Combinaison **fixe** (moyenne, vote)
- Pas d'apprentissage de la combinaison
- Simple mais limité

Stacking

- Combinaison **apprise**
- Le méta-modèle apprend les forces/faiblesses de chaque modèle
- Plus puissant mais risque de surapprentissage

L'Avantage du Stacking

Le méta-modèle peut apprendre que :

- "Modèle 1 est bon quand Modèle 2 hésite"
- "Modèle 3 est fiable quand sa prédiction est > 0.9 "

Attention

Si on entraîne les modèles de base sur **tout** le training set, puis le méta-modèle sur leurs prédictions...

⇒ Le méta-modèle voit des prédictions sur des données **déjà vues** par les modèles de base !

⇒ **Surapprentissage** du méta-modèle.

Solution : Cross-Validation

Utiliser des prédictions **out-of-fold** pour entraîner le méta-modèle.

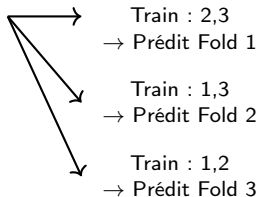
Chaque prédiction d'entraînement provient d'un modèle qui **n'a pas vu** cet exemple.

Stacking avec Cross-Validation

Données



Modèle



Résultat : Prédictions OOF pour tout le training set

Ce processus est répété pour chaque modèle de base.

L'Algorithme de Stacking

Phase 1 : Générer les Meta-Features

Pour chaque modèle de base $m = 1, \dots, M$:

- 1 Faire une cross-validation K -fold
- 2 Pour chaque fold k , entraîner sur les $K - 1$ autres folds, prédire le fold k
- 3 Concaténer les prédictions OOF $\rightarrow$ meta-feature $\hat{y}_m$

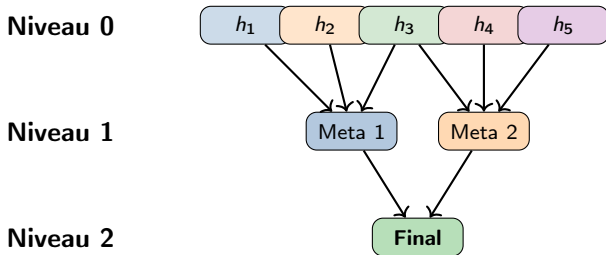
Phase 2 : Entraîner le Méta-Modèle

Créer $X_{meta} = [\hat{y}_1, \hat{y}_2, \dots, \hat{y}_M]$ et entraîner le méta-modèle sur (X_{meta}, y)

Prédiction

Faire prédire les modèles de base, puis le méta-modèle sur leurs sorties.

Stacking Multi-Niveaux



Attention

Plus de niveaux = plus de risque de surapprentissage et complexité.
En pratique, 2 niveaux suffisent généralement.

Méthode	Principe
Voting	Combiner modèles différents par vote
Bagging	Bootstrap + agrégation
Random Forest	Bagging + feature sampling
Boosting	Séquentiel, corriger erreurs
AdaBoost	Pondérer les exemples difficiles
Gradient Boosting	Fit les résidus (gradient de la loss)
Stacking	Méta-modèle sur prédictions

Quand Utiliser Quoi ?

Random Forest

- ✓ Premier choix pour commencer un nouveau problème
- ✓ Données bruitées, features mixtes

Gradient Boosting (XGBoost, LightGBM)

- ✓ Quand RF n'est pas assez performant
- ✓ Données tabulaires en compétition

Stacking

- ✓ Pour les derniers gains de performance
- ✓ Compétitions ML

Conseil Général

Commencer simple (RF), puis complexifier si nécessaire !

Des questions ?